

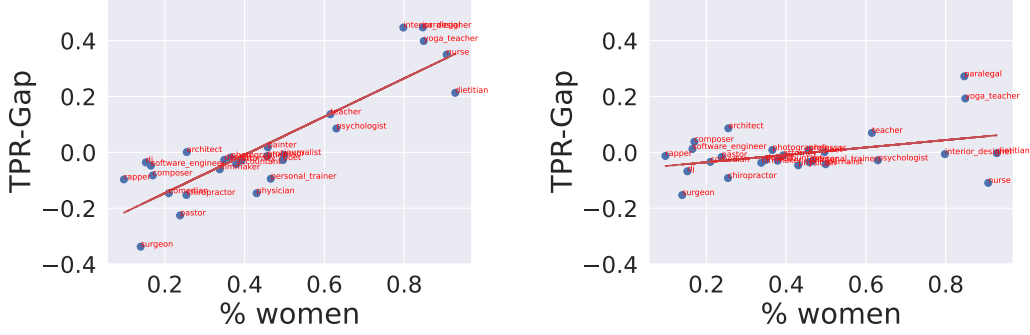


Figure 3: The correlation between $GAP_{female,y}^{TPR}$ and the relative proportion of women in profession y , for BERT embedding, before (left; $R=0.867$) and after (right; $R=0.392$) the projection.

compare our intervention with RLACE [31], which uses gradient-based optimization to solve a linear concept-erasure adversarial game.

Concept erasure results. First, we evaluate the ability of logistic regression classifiers to recover the removed information. The results, presented in Fig. 2, show that our method is the only to achieve random accuracy (perfect erasure) with a small edit, although RLACE (but not INLP) comes close. At the same time, our method is around 2 orders of magnitude faster, and does not require gradient-based optimization.

5.2 Downstream Fairness

How does our intervention affect the behavior of the model on the main classification task of profession prediction? We fit a logistic regression profession-prediction classifier over the projected [CLS] embeddings.

To measure the bias in a classifier, we follow De-Arteaga et al. [6] and use the TPR-GAP measure, which quantifies the bias in a classifier by considering the difference (GAP) in the true positive rate (TPR) between individuals with different protected attributes (e.g. race or gender). We use the notation $GAP_{z,y}^{TPR}$ to denote the TPR-gap in some main-class label y (e.g. “nurse” prediction) for some protected group z (e.g. “female”), we also consider $GAP_z^{TPR,RMS}$, the RMS of the TPR-gap across all professions for a protected group z :

$$GAP_z^{TPR,RMS} = \sqrt{\frac{1}{|C|} \sum_{y \in C} (GAP_{z,y}^{TPR})^2}$$

To calculate the relation between the bias the model exhibits and the bias in the data, we also calculate $\sigma_{(GAP^{TPR}, \%Women)}$, the correlation between the TPR gap in a given profession and the percentage of women in that profession.

Results. The main-task classifier achieves profession-prediction accuracy of 77.3% on the projected embeddings (compared with 79.3% over the original embeddings), indicating that the intervention minimally affects the ability to predict the profession of a person from the embedding of their biography. At the same time, the TPR gap drops significantly from 0.198 to 0.084, indicating a sharp drop in the biased behavior of the profession classifier. Indeed, inspecting the correlation $\sigma_{(GAP^{TPR}, \%Women)}$ between the gap (per profession) and the embedding of women in this profession,

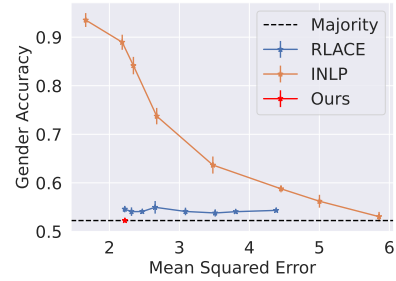


Figure 2: Gender prediction accuracy after bias-removal projection versus the mean squared distance from the original embedding for INLP, RLACE, and LEACE on BERT embeddings.

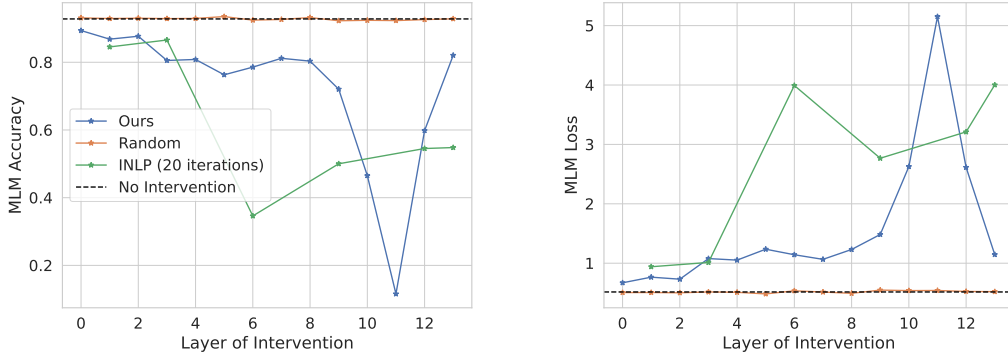


Figure 4: Amnesic probing results on bert-base-uncased.

we see that this correlation plummets from 0.867 to 0.392 after erasure. Re-fitting the main-task logistic regression classifier over the projected embeddings yields a slightly higher main-task accuracy of 78.1%, at the price of significantly increasing the TPR gap to 0.158.⁵

5.3 Revisiting Amnesic Probing

Elazar et al. [10] have introduced the idea of *amnesic probing* as a causal intervention that aims to test the importance of a given concept (e.g. part-of-speech tag) to some main task (e.g. language modeling). They applied Iterative Nullspace Projection (INLP) to remove different concepts from the activations of the model, and assessed the degree to which its behavior changed when performing masked language modeling. Since INLP often requires dozens of iterations to completely erase the concept, its usage in this context raises concerns of collateral damage due to magnitude of the intervention and the non-exhaustive nature of INLP removal. Here, we replicate their experiments on the bert-base-uncased model with our interventions.

Experimental setup. We use part-of-speech (POS) tags as our concept of interest. We collect sentences and their coarse POS tags (“Noun”, “Verb” etc.; 18 in total) from the English Universal Dependencies dataset [27]. We tokenize the sentences with the BERT tokenizer and map each word-piece to the POS tag of the word to which it belongs. We collect the unmasked BERT embeddings for each layer, intervene to linearly erase the POS concept from that layer, and continue the forward pass until the last layer, from which we compute the distribution of the MLM over the vocabulary. Note that in each experiment we intervene on a single layer. We quantify the decrease in accuracy following the intervention, as well as the increase in the loss. We compare with a baseline intervention of a random orthogonal projection whose null space has the same rank as the label space (18). For INLP, we perform 20 iterations. This is needed because INLP does not effectively remove the concept; even after 20 iterations, classification accuracy is above majority accuracy. As a result, INLP reduces the rank of the embedding by 360. By contrast, our method decreases the rank just by 17.

Results. The results are shown in Fig. 4b. Our intervention only mildly changes BERT LM accuracy and loss until layer 8, with the highest drop recorded in layer 11. INLP, in contrast, shows maximum effect at layer 6. Since it removes hundreds of dimensions, it is difficult to attribute this effect to the erasure of the concept. These results suggest that the *causal* effect of the POS concept on the language model is concentrated in layer 11. Interestingly, this stands in contrast with POS linear probing results, which are optimal at earlier layers [38]. As Elazar et al. [10] have noted, probing does not generally correlate with intervention-based analysis techniques.

⁵The softmax probabilities of a multiclass logistic regression classifier can leak the removed information if another classifier is stacked on top of it [33], though this setup is not linear.

6 Concept Scrubbing

Unfortunately, Elazar et al. [10] were forced to limit their interventions to a single layer due to the limitations of INLP. INLP often requires the deletion of several dozen dimensions before linear guarding is achieved—as demonstrated in Figure 2. Kumar et al. [21] show empirically and theoretically that INLP causes needless “collateral damage” to useful parts of the embedding that are orthogonal to the concept being erased. Because of this collateral damage, it’s impossible to apply INLP to multiple layers of a transformer without causing its outputs to collapse into gibberish.

Instead, we would like to erase all linear information about a concept in the activations at *every* layer, which we term **concept scrubbing**. LEACE makes concept scrubbing possible and eminently practical. It causes minimal collateral damage, induces little computational overhead, and the covariance statistics it relies on can be computed in a *streaming* fashion, without ever storing all the hidden states in memory or on disk.

Algorithm. Any intervention on the model at layer ℓ changes the distribution of hidden states at layers $\ell' > \ell$. Because of this, the naive approach of independently fitting LEACE parameters $(\mathbf{P}, \mathbf{b})$ for all layers of the clean model, then applying them all at once, may fail to fully erase the target concept. Instead, we fit LEACE parameters *sequentially*, starting from the first layer and proceeding to the final layer. After we compute $(\mathbf{P}, \mathbf{b})$ for a layer, we immediately use them to scrub the hidden states for that layer, then feed these scrubbed embeddings to the next layer (Algorithm 1).

Algorithm 1 Concept scrubbing

Require: Model with ℓ layers $f = f_\ell \circ \dots \circ f_1$
Require: Design matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$
Require: Label matrix $\mathbf{Z} \in \mathbb{R}^{n \times k}$
Ensure: LEACE parameters for each layer in f

- 1: $\mathbf{H}_1 \leftarrow \text{Embed}(\mathbf{X})$
- 2: $L \leftarrow \text{list}()$
- 3: **for** $l \in 1 \dots \ell$ **do**
- 4: Fit $(\mathbf{P}, \mathbf{b})$ on $\mathbf{H}_l$ and $\mathbf{Z}$
- 5: Append $(\mathbf{P}, \mathbf{b})$ to L
- 6: $\mathbf{H}_l \leftarrow \mathbf{P}(\mathbf{H}_l - \mu_{\mathbf{H}_l}) + \mu_{\mathbf{H}_l}$ (Eq. 1)
- 7: $\mathbf{H}_{l+1} \leftarrow f_l(\mathbf{H}_l)$
- 8: **return** L

Condition	LLaMA			Pythia			
	7B	13B	30B	160M	1.4B	6.9B	12B
No intervention	0.69	0.66	0.62	0.90	0.70	0.64	0.62
Random erasure	0.69	0.66	0.62	0.99	0.72	0.66	0.63
LEACE	1.73	1.84	1.96	2.79	2.25	3.57	3.20
SAL	3.24	3.26	3.16	3.53	3.44	4.17	4.69
unigram entropy	2.90	2.90	2.90	2.66	2.66	2.66	2.66

Table 1: Perplexity in autoregressive language models when removing linearly available part-of-speech information from the input to each transformer layer. Units are bits per UTF-8 byte. The unigram baseline assigns probabilities to tokens based only on their frequency and not on the context.

6.1 Experimental Details

Dataset. For each model family, we use a sample from the respective pretraining distribution: the validation split of the Pile [13] for the Pythia models [2], and the RedPajama replication of the LLaMA pretraining corpus for the LLaMA family [39]. sampling a slice of 2^{22} tokens for fitting the LEACE parameters and another slice of 2^{22} tokens for evaluation. Since neither corpus comes with part-of-speech tags, we use the model from the SpaCy library [19] to automatically generate Universal Dependency tags [23].

Baseline method. We also run concept scrubbing using full-rank SAL [36], which is similar to our method but lacks a bias term and does not adjust for correlations between features (Appendix D).

Architecture. We focus on autoregressive language models. We evaluate our method on EleutherAI’s Pythia 160M, 1.4B, 6.9B, and 12B models [2], and Meta’s LLaMA 7B, 13B, and 30B [39]. We apply concept erasure to the input of each transformer block, immediately after normalization is applied (LayerNorm or RMSNorm).